Commitment-Bound Selective Disclosure for Patient-Held Health Records on Solana and IPFS

M. Jabez Joshwin Paul[1*] and E. Bijolin Edwin[2]

[1*]Division of Computer Science and Engineering, Karunya Institute of Technology and Sciences, Coimbatore, Tamil Nadu, India.
[2]Department of Computer Science and Engineering, Karunya Institute of Technology and Sciences, Coimbatore, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): jabezjoshwin@gmail.com;

Abstract

Purpose: Patients are often asked to prove a simple fact about their health record, such as having received at least a required number of vaccine doses, and end up disclosing the whole record. A zero-knowledge proof can answer the question without revealing the data, but only if the proof is tied to a stored record and to a verification session. We address this for a patient-held record stored on IPFS and anchored on Solana.
Methods: Starting from a prototype in which a server computed the proof from the private value, we store a salted Poseidon commitment to the value in the record's on-chain account, make the same commitment an output of the circuit, add a verifier-chosen challenge as a public input, enforce input ranges inside the circuit, and generate Groth16 proofs in the owner's browser. We implement the design with Circom, snarkjs and Anchor, test it, and measure it on a laptop.
Results: The bound circuit has 339 constraints against 37 for the baseline. A proof takes a median of 89.4 ms to generate in a browser and 13.4 ms to verify. Each record occupies a 212-byte account with a refundable deposit of 0.0023664 SOL, and its creation consumes at most 13,402 compute units.
Conclusion: Binding a threshold proof to an on-chain commitment and proving on the owner's device is practical for this claim. The committed value remains self-asserted; an issuer attestation is the main missing piece.

Keywords: Zero-knowledge proofs, Selective disclosure, Personal health records, Blockchain, Solana, IPFS

1 Introduction

A personal health record is a health record that the individual, rather than a care provider, controls and shares [1, 2]. Putting the individual in control raises a practical question each time the record is used: how much of it must be shown? Many everyday checks need only a yes-or-no fact. A school, an employer or a border officer may need to know that a person has received at least a required number of vaccine doses, and has no need for the dates, the products, or the rest of the record. Handing over the whole document to answer such a question discloses far more than the question requires, which is at odds with the data-minimisation principle found in data-protection law [3].

Blockchains have been studied extensively as a substrate for health records because they give patients and institutions a shared, tamper-evident log without a single operator [4, 5]. Most proposals keep the record itself off-chain, encrypted, and use the chain to manage pointers and permissions [6–11]. In these designs the unit of sharing is still the record or a part of it: a party that is granted access decrypts and reads the data. Zero-knowledge proofs [12] offer a different unit of sharing, namely a statement about the data, and succinct non-interactive constructions [13, 14] make such statements cheap to check.

A zero-knowledge proof about a health value is only useful to a verifier if it is anchored to something. A proof that "I know a number that is at least two" convinces nobody of anything. Three questions have to be answered by the surrounding system: which stored record does the proof speak about, who learned the secret while the proof was being made, and can a proof made for one verifier be shown to another. This paper starts from a small working prototype in which none of the three had a satisfactory answer. In that baseline, the encrypted record was pinned to the InterPlanetary File System (IPFS) and its content identifier was stored on Solana, but the threshold proof was computed by a server that received the private value in clear, and the proof contained nothing that connected it to the stored record or to a particular verification session.

We redesign the proof path to close these gaps and measure what the redesign costs. The contributions are as follows.

1. *A commitment-bound disclosure design.* A salted Poseidon commitment to the private value is stored in the record's on-chain account when the record is created, and the circuit outputs the same commitment. A verifier accepts a proof only if its commitment equals the one read from the chain. A verifier-chosen challenge is a public input of the circuit, which ties each proof to one verification session. Proofs are generated on the owner's device, so no other party sees the private value.
2. *A working implementation* on Solana (using the Anchor framework), IPFS, Circom and snarkjs, with automated tests for the circuit, the on-chain program and the client logic.
3. *A reproducible measurement* of the cost of the design on commodity hardware: circuit size, proving and verification time in Node.js and in a web browser, client-side encryption time, and on-chain storage and execution cost. Every figure reported here is produced by a script included with the implementation.

4. *An explicit account of what the design does not provide*, including the absence of an issuer attestation, the single-party setup, and the information a verifier can extract by repeated queries.

We do not claim a new cryptographic primitive. The building blocks are standard; the contribution is their composition for this use case, an implementation, and measurements that others can repeat.

The rest of the paper is organised as follows. Section 2 introduces the building blocks and Section 3 reviews related work. Section 4 presents the threat model and the design, Section 5 the implementation, and Section 6 the evaluation. Section 7 analyses security and limitations, and Section 8 concludes.

2 Background

2.1 Zero-knowledge succinct arguments

A zero-knowledge proof lets a prover convince a verifier that a statement is true without revealing why it is true [12]. A zk-SNARK is a non-interactive argument of this kind whose proofs are short and fast to verify. Pinocchio [13] made such arguments practical for general computations, and Zerocash [15] and Hawk [16] brought them to blockchains; a survey of their use in that setting is given by Sun et al. [17].

We use the pairing-based argument of Groth [14]. The computation to be proved is written as a rank-1 constraint system (R1CS) over a prime field. A Groth16 proof consists of three group elements, and verification checks one pairing-product equation using three pairings [14]. The scheme needs a structured reference string that is specific to the circuit and is produced by a setup procedure; whoever knows the randomness of that procedure can forge proofs, which is why multi-party procedures for sampling the parameters have been developed [18]. Our implementation works over a Barreto–Naehrig pairing-friendly curve [19], the curve that snarkjs calls `bn128`. Alternatives that avoid a trusted setup exist, for example Bulletproofs [20], whose range proofs fit this kind of statement at the price of larger proofs and slower verification than Groth16.

Circuits are written in Circom [21], a language that compiles a description of signals and constraints into an R1CS and a WebAssembly witness generator. Proving and verification use snarkjs [22]. A known hazard of this style of programming is the *under-constrained* circuit, in which the constraints admit witnesses that the programmer did not intend, so that a malicious prover can obtain an accepting proof for a false statement [23]. This hazard shaped the range checks in Section 4.6.

2.2 Poseidon

Conventional hash functions such as SHA-256 are expensive to express as arithmetic constraints. Poseidon [24] is a hash function designed to operate natively on prime-field elements so that it needs far fewer constraints inside a proof system. We use it as a commitment: the commitment to a value v with random salt s is $C = H(v, s)$, where H is Poseidon with two inputs.

2.3 Solana accounts and program-derived addresses

Solana stores all state in accounts. Every account has the same five fields, among them a data buffer and an owner, and only the program that owns an account may modify its data [25]. An account must hold a refundable minimum balance proportional to its size to remain on chain [25]. A program-derived address (PDA) is an account address computed deterministically from a program identifier and a list of seeds; it does not lie on the Ed25519 curve, so no private key exists for it [26]. A transaction is limited to 1,232 bytes [27]; it pays a base fee of 5,000 lamports per signature, and each instruction has a default budget of 200,000 compute units [28]. One SOL is 10^9 lamports. Anchor is a framework that generates account validation and serialisation code for Solana programs from declarative annotations [29].

2.4 IPFS

IPFS is a peer-to-peer storage network in which data is addressed by a content identifier (CID) derived from a cryptographic hash of the data itself [30, 31]. Because the address is a function of the content, anyone who retrieves the data can check that it has not been altered. Content remains retrievable only while some node keeps it; *pinning* services keep content on behalf of users [32]. IPFS provides no confidentiality: anyone who knows a CID can request the content, so private data must be encrypted before it is added.

2.5 Password-based authenticated encryption

AES in Galois/Counter Mode (GCM) is an authenticated encryption mode; NIST recommends a 96-bit initialisation vector for it [33]. PBKDF2 derives a symmetric key from a passphrase by iterating a pseudorandom function over the passphrase and a salt [34, 35]. Both are available in browsers through the Web Cryptography API [36]. The iteration count determines the cost of each passphrase guess; NIST SP 800-132 asks for a count as large as is feasible [35], and current OWASP guidance recommends 600,000 iterations for PBKDF2 with HMAC-SHA-256 [37].

3 Related Work

Blockchain-based health record management

Zyskind et al. [7] proposed keeping personal data off-chain and using a blockchain as an access-control manager. MedRec [6] applied the idea to medical records, using smart contracts to manage permissions over records that remain with providers. Yue et al. [38] described a gateway application that lets patients own and control their data. Ancile [8] combines smart contracts with further cryptographic techniques for access control and interoperability, and MeDShare [9] records data-sharing actions among cloud custodians on a blockchain to provide provenance and auditing. FHIR-Chain [39] builds a blockchain-based architecture around the HL7 FHIR standard for sharing clinical data. Shahnaz et al. [10] and Tanwar et al. [40] present frameworks for electronic health records with access rules enforced on a blockchain. Sun et

al. [11] store encrypted medical records in IPFS and control access with ciphertext-policy attribute-based encryption. Surveys of the area identify privacy, scalability and regulatory compliance as open challenges [4, 5, 41].

These systems answer the question of *who may read a record*. Once access is granted, the reader obtains the data. Our work addresses a complementary question, namely how an owner can let a verifier check a predicate over a record without granting read access at all.

Zero-knowledge proofs and credentials in health applications

Sharma et al. [42] combine zero-knowledge proofs with proxy re-encryption in a blockchain framework for managing access to health records and funds. Selective disclosure has a long history in anonymous credential systems [43], and the W3C Verifiable Credentials data model [44] standardises a format for issuer-signed credentials in which a holder can choose what to disclose, including through zero-knowledge mechanisms. Those systems rest on an issuer who signs the attributes. Our prototype has no issuer; the on-chain commitment fixes a value at a point in time but does not attest that the value is true. We return to this limitation in Section 7.2.

Positioning

We are not aware of a published measurement of a commitment-bound threshold proof for health records on Solana with client-side proving, but we have not carried out a systematic review and make no claim of being first. We do not compare our measurements numerically with the systems above: they run on different ledgers, solve different problems and report different metrics, so a side-by-side table of numbers would not be meaningful.

4 System Design

4.1 Roles and threat model

The system has three roles. The *owner* holds a health record and a wallet key. A *verifier* wants to know whether a numeric field of the record, here the vaccination count, is at least some threshold. The *infrastructure* consists of the Solana ledger, the IPFS network with a pinning service, and a web server that delivers the application code.

We consider the following adversaries.

- *Chain and storage observers.* Everything written to Solana and to IPFS is public and permanent for practical purposes. Observers try to learn the contents of records from it.
- *A curious verifier.* The verifier follows the protocol but tries to learn more than the truth of the predicate.
- *A dishonest prover.* A party tries to make a verifier accept a statement that is false for the record at a given address, or to reuse a proof it obtained from someone else.
- *A curious service operator.* The operators of the pinning service and of any auxiliary server try to learn private values from what passes through them.

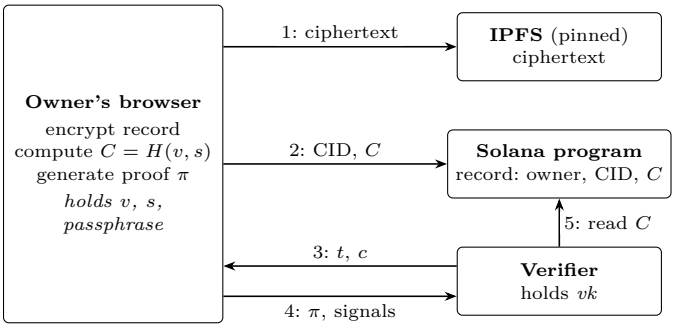


Fig. 1 Architecture and data flow. Steps 1–2 create a record; steps 3–5 are one disclosure. Step 4 carries the record address, the proof π and the public signals. The private count and salt never leave the owner's browser.

We assume that the owner's device and browser are not compromised, that the application code and the verification key reach each party unmodified, that the setup of the proof system was honest, and that the cryptographic primitives are secure. The owner is *not* assumed to be truthful about the data: the design makes no claim that the committed value is medically correct. Denial of service is out of scope.

4.2 Design goals

- *G1, record confidentiality.* Observers and operators learn nothing about a record's contents beyond its size.
- *G2, minimal disclosure.* From an accepted proof a verifier learns that the count is at least the threshold and nothing else about the count.
- *G3, binding to a record.* An accepted proof is about the value that was committed when the record at the stated address was created.
- *G4, session freshness.* A proof made for one challenge is not accepted for another.
- *G5, local proving.* No party other than the owner's device handles the private value or the salt in clear.

4.3 Architecture

Figure 1 shows the components. The owner's browser performs all operations on private data: encryption, commitment and proving. The pinning service receives only ciphertext, through a server-side route that holds the service credential. The Solana program stores one small account per record. The verifier needs only the verification key, read access to the chain, and the proof.

4.4 Record creation

Let v be the private count. To create a record the owner's browser

1. samples a salt s of 248 random bits and stores it as a field of the record;
2. computes the commitment $C = H(v, s)$;

3. derives a 256-bit key from the owner's passphrase with PBKDF2 and a fresh 128-bit salt, encrypts the record (including s) with AES-256-GCM under a fresh 96-bit initialisation vector, and uploads the result to IPFS, obtaining a CID;
4. sends an `init_record` transaction that creates the record account and stores the owner's public key, the CID, a client-supplied timestamp and C.

The account address is a PDA derived from the seeds (`"record"`, owner key, timestamp), so each upload creates a new account and existing accounts are never overwritten. The program stores C as 32 opaque bytes; it neither interprets nor checks it. Keeping the salt inside the encrypted record means that the owner needs only the passphrase to prove again later from any device.

4.5 Disclosure protocol

A disclosure involves the following steps.

1. The verifier chooses a threshold t and a fresh random challenge c, and sends both to the owner.
2. The owner's browser generates a Groth16 proof π for the relation in Section 4.6 with private inputs (v, s) and public inputs (t, c), and sends the verifier the record address, π and the public signals (b, C', t', c').
3. The verifier reads the commitment C from the record account at that address, directly from a Solana node.
4. The verifier accepts if and only if

$$\begin{gathered} C' = C, \qquad t' = t, \qquad c' = c, \qquad b = 1, \\ \mathsf{Verify}\big(vk,\ (b, C', t', c'),\ \pi\big) = 1. \end{gathered} \tag{1}$$

The verifier never takes the commitment from the prover's message as given; it is compared with the on-chain value. Each failed condition yields a distinct rejection reason in the implementation, which makes misuse easier to diagnose.

4.6 The circuit

Let $\mathbb{F}_p$ be the scalar field of the curve and $n = 32$. The circuit enforces the relation

$$\begin{aligned} \mathcal{R} = \big\{\, ((b, C, t, c),\ (v, s)) \ :\ & v, t \in [0, 2^n), \\ & C = H(v, s),\ \ b = 1 \text{ if } v \geq t,\ \ b = 0 \text{ otherwise} \,\big\} \end{aligned} \tag{2}$$

with (v, s) private and (b, C, t, c) public. The comparison is computed as $t < v + 1$ with a comparator on $n + 1$ bits, because $v + 1$ can reach 2^n.

Two details matter for soundness. First, the comparator from the Circom standard library is correct only when both operands fit in the stated number of bits. The baseline circuit relied on that precondition without enforcing it. The new circuit decomposes v and t into n bits each, so the bound is part of the proved statement rather than an assumption about the prover. Second, the challenge c does not take part in the predicate, and a public input that appears in no constraint may not be bound by the

proof. The circuit therefore includes one multiplication constraint, $c \cdot c$, whose only purpose is to tie c into the constraint system.

The bit b is an output rather than a hard constraint. A proof with $b = 0$ is a valid proof that the count is *below* the threshold; the verifier's rule (1) requires $b = 1$.

5 Implementation

The prototype has four parts.

Circuit

The circuit is about fifty lines of Circom and uses the bit-decomposition, comparator and Poseidon templates of circomlib. It is compiled with full constraint simplification. The proving and verification keys come from a single-contributor setup run with snarkjs (Section 7.2).

On-chain program

The program is written in Rust with Anchor 0.30.1. It has one instruction, `init_record`, and one account type holding the owner key (32 bytes), the CID as a string of at most 128 bytes, a 64-bit timestamp and the 32-byte commitment. With Anchor's 8-byte type discriminator and the 4-byte string length prefix the account occupies 212 bytes. A CID longer than 128 bytes is rejected with a dedicated error. The owner must sign and pays for the account.

Client

The web client is a Next.js application. It uses the Web Cryptography API for PBKDF2 (HMAC-SHA-256, 120,000 iterations) and AES-256-GCM, and loads snarkjs, the WebAssembly witness generator (1.75 MB) and the proving key (206.9 kB) to prove in the browser. The commitment is computed outside the circuit with an independent Poseidon implementation, and a test checks that the two agree. The client contains an owner view, which creates records and generates proofs, and a verifier view, which reads a record account by address and applies rule (1). The verifier view needs no wallet.

Auxiliary services

Uploads to the pinning service pass through a server-side route that holds the service credential; it sees ciphertext only. A small optional HTTP service verifies a proof against the verification key for clients that cannot run snarkjs. Unlike the baseline's proof service, it never receives private inputs.

Development method and use of AI tools

Claude (Anthropic), a large language model used through the Claude Code tool, assisted in this work. It was used to draft the text of all sections of the manuscript; to implement the commitment and challenge extension of the circuit, the client-side proving and verification code, the test suites and the benchmark scripts; and to generate the tables and the timing figure from the recorded measurement files. Every

Table 1 Size of the constraint system and of the artifacts.

Quantity	Baseline	Bound
R1CS constraints	37	339
Wires	39	341
Private inputs	1	2
Public signals	2	4
Witness generator (bytes)	36,469	1,750,062
Proving key (bytes)	22,333	206,869
Verification key, JSON (bytes)	3,109	3,476
Proof, JSON (bytes)	721	721
Public signals, JSON (bytes)	9	120

numerical result reported here was obtained by executing the code on the hardware described in Section 6; none was produced by the model. Each reference was checked against its publisher, standards-body or DOI registry record. The authors reviewed the manuscript and the code and take full responsibility for the content.

6 Evaluation

The evaluation answers three questions. What does binding the proof to the record cost in circuit size and time? Is proving on the owner's device practical? What does a record cost on chain?

6.1 Setup and method

All measurements were taken on one laptop: an Apple M1 system with 8 cores (4 performance and 4 efficiency) and 8 GB of memory, running macOS 26.6.2. Software versions were Node.js 24.10.0, Circom 2.2.3, circomlib 2.0.5, snarkjs 0.7.6, Anchor 0.30.1 and the Solana tool suite 3.1.13.

We compare two circuits. *Baseline* is the threshold-only circuit of the original prototype, with one private input and no commitment, challenge or range checks. *Bound* is the circuit of Section 4.6. Timings in Node.js are wall-clock times over 100 runs after 5 unmeasured warm-up runs. Browser timings were taken in Chromium 156 on the same machine with a production build of the client, over 30 runs after 3 warm-up runs. Encryption timings use the client's own encryption functions over 30 runs. On-chain costs were measured on a local Solana test validator with the program loaded, over 20 transactions with a 59-character CID. We report medians with the 5th and 95th percentiles.

We did not measure confirmation latency on a public Solana cluster or upload latency to a pinning service; both depend on network conditions and on third-party services, and the numbers below should not be read as including them.

Table 2 Time in milliseconds: median (5th–95th percentile).

Operation	Baseline	Bound
Node.js, $n = 100$		
Witness generation	5.3 (4.4–6.2)	30.6 (30.1–31.3)
Groth16 proving	22.9 (20.1–26.1)	74.8 (67.9–78.9)
Witness + proving	30.2 (28.8–32.9)	108.4 (103.4–115.5)
Verification	12.3 (12.1–14.0)	13.4 (13.2–13.8)
Browser (Chromium 156), $n = 30$		
Witness + proving	—	89.4 (82.8–101.1)
Verifier check	—	14.1 (13.6–15.0)

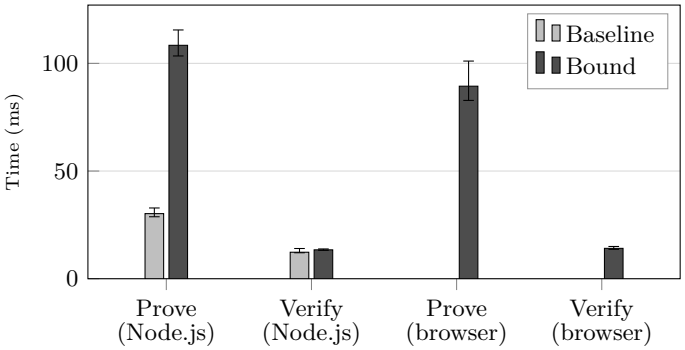


Fig. 2 Median proving and verification time. Whiskers show the 5th and 95th percentiles. The baseline was not run in the browser because it proved on a server.

6.2 Circuit size

Table 1 compares the two circuits. Binding raises the number of constraints from 37 to 339, a factor of 9.2. Most of the increase comes from the commitment: a circuit containing only the two-input Poseidon hash compiles to 240 constraints, about 71% of the total. The range checks and the wider comparator account for most of the rest. Compiler simplification matters: without it the same circuit has 622 constraints, of which 280 are linear and are eliminated by simplification.

The proof is the same size in both cases, as expected for Groth16, whose proofs always consist of three group elements [14]; its JSON encoding is 721 bytes. The public signals grow because they now include the commitment and the challenge. The artifact that grows most is the witness generator, from 36.5 kB to 1.75 MB, because it embeds the Poseidon round constants. This is a one-time download for the owner's browser and can be cached.

6.3 Proving and verification time

Table 2 and Fig. 2 give the timings. In Node.js, witness generation plus proving takes 108.4 ms for the bound circuit against 30.2 ms for the baseline, a factor of 3.6 for 9.2

Table 3 Client-side encryption and decryption (median, ms). "Stored" is the size of the JSON object pinned to IPFS.

Plaintext (bytes)	Stored (bytes)	Encrypt (ms)	Decrypt (ms)
228	451	23.4	23.4
1,024	1,511	23.5	23.4
16,384	21,991	23.7	23.6
65,536	87,527	24.6	23.8
262,144	349,671	27.4	24.8

times as many constraints. Verification time is practically unchanged, 13.4 ms against 12.3 ms, because Groth16 verification cost depends on the number of public inputs and not on the size of the circuit.

In the browser, the complete proving step has a median of 89.4 ms and a 95th percentile of 101.1 ms once the artifacts are loaded. The first proof after a page load took 257.1 ms in our run, which includes fetching and compiling the witness generator. The verifier's complete check, including the comparisons of rule (1), has a median of 14.1 ms. For this circuit, proving on the owner's device is therefore well within interactive latency on a laptop, and the server-side prover of the baseline is not needed for performance reasons. We did not measure phones or low-end devices, where proving will be slower.

6.4 Client-side encryption

Table 3 shows encryption and decryption times for records of increasing size. Key derivation alone takes 23.3 ms and dominates: the sample record encrypts in 23.4 ms and a record of 256 KiB in 27.4 ms. The stored object is larger than the plaintext because the ciphertext is Base64-encoded and wrapped with its salt, initialisation vector and algorithm labels; the sample record of 228 bytes becomes 451 bytes.

That key derivation takes only about 23 ms is not purely good news. The same cost applies to each guess of an attacker who downloads the public ciphertext, and the configured 120,000 iterations are below the 600,000 that OWASP currently recommends for this function [37]. We discuss this in Section 7.2.

6.5 On-chain cost

Table 4 lists the cost of `init_record`. The record account occupies 212 bytes, 32 more than in the baseline, and requires a refundable deposit of 2,366,400 lamports (0.0023664 SOL). This matches the documented formula for the minimum balance [25]. Storing the commitment adds 222,720 lamports to the deposit compared with the baseline layout. The transaction is 347 bytes, 28% of the 1,232-byte limit [27], and pays the base fee of 5,000 lamports for its single signature.

The instruction consumed between 7,402 and 13,402 compute units, far below the default budget of 200,000 [28]. The observed values differ by exact multiples of 1,500

Table 4 On-chain cost of creating one record (local test validator).

Quantity	Value
Record account size (bytes)	212
Rent-exempt deposit (lamports)	2,366,400
Transaction size (bytes)	347
Compute units consumed	7,402–13,402
Transaction fee (lamports)	5,000
Total debit to owner (lamports)	2,371,400

units. We attribute this to the search for the PDA bump seed, which depends on the owner key and timestamp and therefore varies between records; we did not instrument the runtime to confirm the attribution. On the local validator a transaction reached the *confirmed* commitment level in a median of 480 ms; this figure describes the test setup only.

The on-chain cost is independent of the size of the health record, since the chain stores only the CID and the commitment. Proof verification costs nothing on chain because it happens off chain.

6.6 Functional validation

The implementation has 28 automated tests. Eleven exercise the circuit: correct outputs above, at and below the threshold; the largest 32-bit values; agreement of the in-circuit commitment with an independent Poseidon implementation; rejection of out-of-range inputs at witness generation; and rejection of proofs whose public commitment, challenge or validity bit has been altered. Six run the program on a local validator: correct storage of all fields, the exact account size, acceptance of a 128-byte CID and rejection of a 129-byte one, rejection of a duplicate record, and rejection of a record whose stated owner did not sign. Eleven cover the client logic, including each rejection reason of rule (1). In addition, the verifier view was exercised interactively in a browser against a record on the local validator: a matching proof was accepted, and the same proof was rejected after changing the expected challenge and after raising the expected threshold. The upload path was tested through the program's client library; the wallet extension's signing dialogue in the browser is not covered by automated tests.

7 Security Analysis and Limitations

7.1 How the goals are met

The arguments in this subsection are informal and rest on the assumptions of Section 4.1.

G1, record confidentiality. Only the AES-GCM ciphertext, its salt and its initialisation vector are stored on IPFS, and only the CID and the commitment are stored

on chain. Each record is encrypted under a key derived with a fresh salt, so an initialisation vector is never reused under the same key. Confidentiality reduces to the security of AES-GCM and to the strength of the passphrase.

G2, minimal disclosure. Groth16 is zero-knowledge [14], so the proof reveals nothing beyond the public signals (b, C, t, c). The threshold and challenge were chosen by the verifier. The commitment $C = H(v, s)$ is already public on chain; the count has very few plausible values, so without a salt it could be recovered by trying each one, and the 248-bit random salt prevents this. We treat $H(v, \cdot)$ with a uniformly random second input as hiding v, which is a heuristic assumption about Poseidon and not a proved property.

G3, binding to a record. If the verifier accepts, then by the knowledge soundness of Groth16 the prover knows (v, s) with $H(v, s) = C$ and $v \geq t$, where C is the value read from the chain. The record account cannot be modified after creation, since the program has no instruction that writes to an existing account. Finding a second opening (v', s') of the same commitment with $v' \neq v$ would be a collision in Poseidon. The proof is therefore about the value committed at creation time. The range checks close the under-constrained comparison discussed in Section 4.6.

G4, session freshness. The challenge is a public input, so a proof verifies only together with the challenge it was made for. A verifier that draws a fresh challenge for each session will reject a proof recorded in an earlier session. Tests confirm that altering the challenge, the commitment or the validity bit of a genuine proof makes verification fail.

G5, local proving. Witness generation and proving run in the owner's browser. The verification service and the pinning route receive no private input.

7.2 Limitations

We list the limitations of the design and of the prototype in what we judge to be decreasing order of importance.

The data is self-asserted. Nothing prevents an owner from committing to a count that is false. The system proves consistency with a value fixed in the past, not correctness of that value. A deployment would need an issuer, such as a clinic, to sign the commitment or the underlying attributes, with the signature checked by the verifier or inside the circuit, as in credential systems [43, 44].

The proof is not bound to the presenter. Anyone who knows (v, s) can prove. The challenge stops later replay but not a relay in which an intermediary forwards a verifier's challenge to the real owner and passes the answer back as its own. Since the record account names the owner's public key, the verifier could additionally require a signature on the challenge under that key; the prototype does not do this.

Repeated queries leak the value. Each accepted or rejected proof tells the verifier one bit. A verifier, or several colluding verifiers, who obtain answers for different thresholds can narrow down the count, in the worst case exactly. The owner decides which requests to answer, but the prototype offers no policy support for this.

Single-party setup. The proving key was generated by one contributor on a development machine. Anyone who retained the setup randomness could forge proofs. A

real deployment needs a multi-party ceremony [18] or a proof system without a trusted setup [20].

Passphrase-based encryption of public ciphertext. The ciphertext is public and cannot be recalled, so an attacker can test passphrases offline without limit. The configured iteration count is below current guidance [37], and a leaked passphrase compromises the record permanently. A memory-hard key derivation function or a key held by the wallet would be stronger.

Linkability and metadata. Records are linked to the owner's wallet address, and their creation times and number are public. All proofs about one record carry the same commitment and are therefore linkable to each other and to that record.

No revocation or erasure. The program cannot update or close a record, and content on IPFS persists while any node keeps it. This conflicts with a right to erasure such as that of Article 17 of the GDPR [3] if the ciphertext or the commitment is considered personal data, a question we do not resolve here.

Off-chain verification. Proofs are verified by the verifier's own software. The verifier must obtain the correct verification key through a channel it trusts; the prototype serves it with the application. The timestamp in a record is supplied by the client and is not checked against the cluster clock.

Scope of the evaluation. One claim type, one machine, a local validator, no public-cluster or pinning-service latency, no mobile devices and no user study.

7.3 Future work

The most valuable extension is issuer attestation, which would turn the self-asserted commitment into a credential. Binding proofs to the owner's wallet key, a multi-party setup, and anchoring a hash of the verification key on chain address the next three limitations. The circuit generalises directly to other numeric predicates and to several attributes committed together; measuring how proving time grows with the number of attributes on phones is the natural next experiment. Verifying the proof inside the Solana program would let other programs act on a disclosure, at a compute cost that remains to be measured.

8 Conclusion

A zero-knowledge proof about a health record is only as meaningful as its connection to a stored record and to a verification session. Starting from a prototype that lacked both connections and revealed the private value to a server, we added a salted Poseidon commitment stored on Solana, a verifier-chosen challenge, enforced range checks and client-side proving. On a laptop the resulting circuit has 339 constraints, proves in about 89.4 ms in a browser and verifies in about 13.4 ms, and each record costs a 212-byte account with a refundable deposit of 0.0023664 SOL. The design gives a verifier a yes-or-no answer about a committed value without access to the record. It does not establish that the committed value is true, and we have set out that and the other limitations that stand between this prototype and a deployable system.

Declarations

Funding. [AUTHORS TO CONFIRM: state the funding source, or that no funding was received.]

Competing interests. [AUTHORS TO CONFIRM: state any competing interests, or that the authors have none to declare.]

Ethics approval. This work uses synthetic sample data only. No human participants, patient data or animals were involved.

Data and code availability. The source code, the test suites, the benchmark scripts and the recorded measurement files from which every table and figure in this article is generated are in the project repository. [AUTHORS TO CONFIRM: give the public repository address and the commit that matches this manuscript.]

Author contributions. [AUTHORS TO CONFIRM: describe each author's contribution.]

References

[1] Tang, P.C., Ash, J.S., Bates, D.W., Overhage, J.M., Sands, D.Z.: Personal health records: Definitions, benefits, and strategies for overcoming barriers to adoption. Journal of the American Medical Informatics Association **13**(2), 121–126 (2006) https://doi.org/10.1197/jamia.M2025

[2] Roehrs, A., da Costa, C.A., da Rosa Righi, R., de Oliveira, K.S.F.: Personal health records: A systematic literature review. Journal of Medical Internet Research **19**(1), 13 (2017) https://doi.org/10.2196/jmir.5876

[3] European Parliament and Council of the European Union: Regulation (EU) 2016/679 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data (General Data Protection Regulation). Official Journal of the European Union (2016). https://eur-lex.europa.eu/eli/reg/2016/679/oj

[4] Kuo, T.-T., Kim, H.-E., Ohno-Machado, L.: Blockchain distributed ledger technologies for biomedical and health care applications. Journal of the American Medical Informatics Association **24**(6), 1211–1220 (2017) https://doi.org/10.1093/jamia/ocx068

[5] McGhin, T., Choo, K.-K.R., Liu, C.Z., He, D.: Blockchain in healthcare applications: Research challenges and opportunities. Journal of Network and Computer Applications **135**, 62–75 (2019) https://doi.org/10.1016/j.jnca.2019.02.027

[6] Azaria, A., Ekblaw, A., Vieira, T., Lippman, A.: MedRec: Using blockchain for medical data access and permission management. In: 2016 2nd International Conference on Open and Big Data (OBD), pp. 25–30 (2016). https://doi.org/10.1109/OBD.2016.11

[7] Zyskind, G., Nathan, O., Pentland, A.: Decentralizing privacy: Using blockchain to protect personal data. In: 2015 IEEE Security and Privacy Workshops, pp. 180–184 (2015). https://doi.org/10.1109/SPW.2015.27

[8] Dagher, G.G., Mohler, J., Milojkovic, M., Marella, P.B.: Ancile: Privacy-preserving framework for access control and interoperability of electronic health records using blockchain technology. Sustainable Cities and Society **39**, 283–297 (2018) https://doi.org/10.1016/j.scs.2018.02.014

[9] Xia, Q., Sifah, E.B., Asamoah, K.O., Gao, J., Du, X., Guizani, M.: MeDShare: Trust-less medical data sharing among cloud service providers via blockchain. IEEE Access **5**, 14757–14767 (2017) https://doi.org/10.1109/ACCESS.2017.2730843

[10] Shahnaz, A., Qamar, U., Khalid, A.: Using blockchain for electronic health records. IEEE Access **7**, 147782–147795 (2019) https://doi.org/10.1109/ACCESS.2019.2946373

[11] Sun, J., Yao, X., Wang, S., Wu, Y.: Blockchain-based secure storage and access scheme for electronic medical records in IPFS. IEEE Access **8**, 59389–59401 (2020) https://doi.org/10.1109/ACCESS.2020.2982964

[12] Goldwasser, S., Micali, S., Rackoff, C.: The knowledge complexity of interactive proof systems. SIAM Journal on Computing **18**(1), 186–208 (1989) https://doi.org/10.1137/0218012

[13] Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. In: 2013 IEEE Symposium on Security and Privacy, pp. 238–252 (2013). https://doi.org/10.1109/SP.2013.47

[14] Groth, J.: On the size of pairing-based non-interactive arguments. In: Advances in Cryptology – EUROCRYPT 2016. Lecture Notes in Computer Science, pp. 305–326. Springer, Berlin, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49896-5_11

[15] Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from Bitcoin. In: 2014 IEEE Symposium on Security and Privacy, pp. 459–474 (2014). https://doi.org/10.1109/SP.2014.36

[16] Kosba, A., Miller, A., Shi, E., Wen, Z., Papamanthou, C.: Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In: 2016 IEEE Symposium on Security and Privacy (SP), pp. 839–858 (2016). https://doi.org/10.1109/SP.2016.55

[17] Sun, X., Yu, F.R., Zhang, P., Sun, Z., Xie, W., Peng, X.: A survey on zero-knowledge proof in blockchain. IEEE Network **35**(4), 198–205 (2021) https://

doi.org/10.1109/MNET.011.2000473

[18] Ben-Sasson, E., Chiesa, A., Green, M., Tromer, E., Virza, M.: Secure sampling of public parameters for succinct zero knowledge proofs. In: 2015 IEEE Symposium on Security and Privacy, pp. 287–304 (2015). https://doi.org/10.1109/SP.2015.25

[19] Barreto, P.S.L.M., Naehrig, M.: Pairing-friendly elliptic curves of prime order. In: Selected Areas in Cryptography. Lecture Notes in Computer Science, pp. 319–331. Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/11693383_22

[20] Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE Symposium on Security and Privacy (SP), pp. 315–334 (2018). https://doi.org/10.1109/SP.2018.00020

[21] Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J.L., Rubio, A., Baylina, J.: Circom: A circuit description language for building zero-knowledge applications. IEEE Transactions on Dependable and Secure Computing **20**(6), 4733–4751 (2023) https://doi.org/10.1109/TDSC.2022.3232813

[22] iden3: snarkjs: zkSNARK Implementation in JavaScript and WASM. Software repository. Accessed 13 November 2026. https://github.com/iden3/snarkjs

[23] Pailoor, S., Chen, Y., Wang, F., Rodríguez, C., Geffen, J.V., Morton, J., Chu, M., Gu, B., Feng, Y., Dillig, I.: Automated detection of under-constrained circuits in zero-knowledge proofs. Proceedings of the ACM on Programming Languages **7**(PLDI), 1510–1532 (2023) https://doi.org/10.1145/3591282

[24] Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 519–535 (2021). https://www.usenix.org/conference/usenixsecurity21/presentation/grassi

[25] Solana Foundation: Accounts. Solana Documentation. Accessed 13 November 2026. https://solana.com/docs/core/accounts

[26] Solana Foundation: Program Derived Addresses (PDAs). Solana Documentation. Accessed 13 November 2026. https://solana.com/docs/core/pda

[27] Solana Foundation: Transactions. Solana Documentation. Accessed 13 November 2026. https://solana.com/docs/core/transactions

[28] Solana Foundation: Fees. Solana Documentation. Accessed 13 November 2026. https://solana.com/docs/core/fees

[29] Anchor Framework Documentation. anchor-lang.com. Accessed 13 November

2026. https://www.anchor-lang.com/docs

[30] Benet, J.: IPFS – Content Addressed, Versioned, P2P File System. arXiv preprint arXiv:1407.3561 (2014). https://arxiv.org/abs/1407.3561

[31] Trautwein, D., Raman, A., Tyson, G., Castro, I., Scott, W., Schubotz, M., Gipp, B., Psaras, Y.: Design and evaluation of IPFS: A storage layer for the decentralized web. In: Proceedings of the ACM SIGCOMM 2022 Conference, pp. 739–752 (2022). https://doi.org/10.1145/3544216.3544232

[32] Daniel, E., Tschorsch, F.: IPFS and friends: A qualitative comparison of next generation peer-to-peer data networks. IEEE Communications Surveys & Tutorials **24**(1), 31–52 (2022) https://doi.org/10.1109/COMST.2022.3143147

[33] Dworkin, M.: Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology (2007). https://doi.org/10.6028/NIST.SP.800-38D

[34] Moriarty, K., Kaliski, B., Rusch, A.: PKCS #5: Password-based cryptography specification version 2.1. RFC 8018, RFC Editor (2017). https://doi.org/10.17487/RFC8018

[35] Turan, M.S., Barker, E., Burr, W., Chen, L.: Recommendation for password-based key derivation: Part 1: Storage applications. NIST Special Publication 800-132, National Institute of Standards and Technology (2010). https://doi.org/10.6028/NIST.SP.800-132

[36] Watson, M.: Web Cryptography API. W3C Recommendation (2017). https://www.w3.org/TR/2017/REC-WebCryptoAPI-20170126/

[37] OWASP Foundation: Password Storage Cheat Sheet. OWASP Cheat Sheet Series. Accessed 13 November 2026. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[38] Yue, X., Wang, H., Jin, D., Li, M., Jiang, W.: Healthcare data gateways: Found healthcare intelligence on blockchain with novel privacy risk control. Journal of Medical Systems **40**(10), 218 (2016) https://doi.org/10.1007/s10916-016-0574-6

[39] Zhang, P., White, J., Schmidt, D.C., Lenz, G., Rosenbloom, S.T.: FHIRChain: Applying blockchain to securely and scalably share clinical data. Computational and Structural Biotechnology Journal **16**, 267–278 (2018) https://doi.org/10.1016/j.csbj.2018.07.004

[40] Tanwar, S., Parekh, K., Evans, R.: Blockchain-based electronic healthcare record system for healthcare 4.0 applications. Journal of Information Security and Applications **50**, 102407 (2020) https://doi.org/10.1016/j.jisa.2019.102407

[41] Zhang, R., Xue, R., Liu, L.: Security and privacy on blockchain. ACM Computing Surveys **52**(3), 1–34 (2020) https://doi.org/10.1145/3316481

[42] Sharma, B., Halder, R., Singh, J.: Blockchain-based interoperable healthcare using zero-knowledge proofs and proxy re-encryption. In: 2020 International Conference on COMmunication Systems & NETworkS (COMSNETS), pp. 1–6 (2020). https://doi.org/10.1109/COMSNETS48256.2020.9027413

[43] Camenisch, J., Lysyanskaya, A.: An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In: Advances in Cryptology – EUROCRYPT 2001. Lecture Notes in Computer Science, pp. 93–118. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-44987-6_7

[44] Sporny, M., Thibodeau Jr, T., Herman, I., Cohen, G., Jones, M.B.: Verifiable Credentials Data Model v2.0. W3C Recommendation (2025). https://www.w3.org/TR/vc-data-model-2.0/